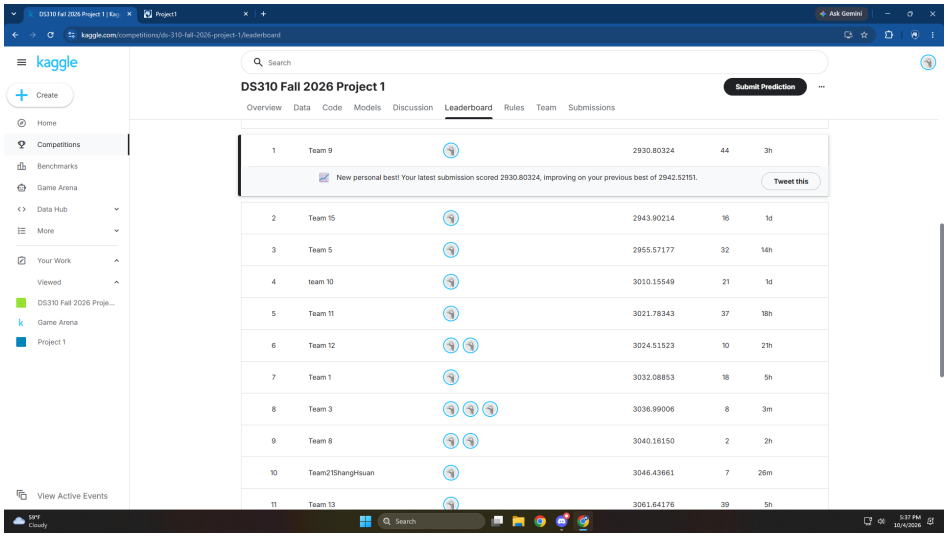
DS 310 Project 1: Regression

Team and account	Name and email	Date
Team 9 / shephmic77	Michael Shepherd / mrs7065@psu.edu	October 4, 2026

Competition and final score

The goal was to predict a number for each test row using the training data. Kaggle used mean squared error, or MSE, to score the predictions. MSE is the average of the squared prediction errors. A lower score is better. Large errors matter more because they are squared.

My final model was a blend of regression models with small changes to its predictions. Its public MSE was **2930.80**. The screenshot shows **Team 9 in first place** with 44 submissions. This was the public rank shown on October 4. The private score was not available.



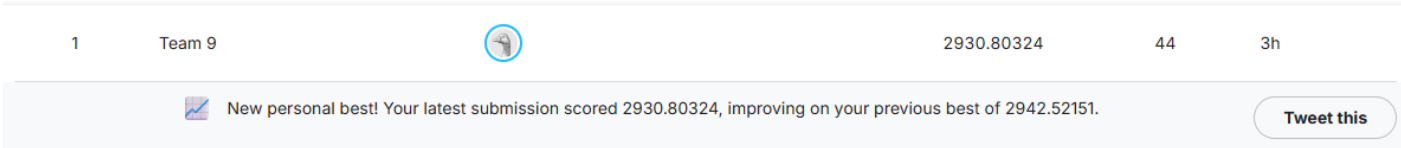


Figure 1. My public leaderboard result. The Team 9 row is enlarged below the screenshot.

Result	MSE
Earlier signed model blend	2942.52
Final signed model blend	2930.80
Improvement	11.72
Local five fold check	3483.96

I started with ridge regression. I tried other models, but the most useful changes were to the features, training weights, and blend weights. The rest of this report explains those choices. Scores are rounded to two decimal places. The exact final Kaggle score was 2930.80324.

1. Data and features

Data	What it contains
Training inputs	242 rows and 64 columns
Training outputs	242 target values, from 25 to 341
Test inputs	200 rows and the same 64 columns
Sample submission	The row IDs and required output format

There were no missing values. I kept all rows and checked that the training and test columns were in the same order. The sample output values were placeholders, so I did not use them as labels. I used only the supplied data. The columns had numbered names, so I did not assume what they meant.

Preparing the data

Some models used the first 10 columns. Others used all 64. For most linear models, I changed each feature to have a training mean of zero and a standard deviation of one. I used those same training values to scale new rows. Trees used the original values. Any prediction limits also came from the training targets.

Two feature sets for weighted ridge

The **smaller feature set** used columns **2, 3, 4, 5, 8, 9, 10, 20, 43, 11, 24**, in that order. I added four features: column 9 raised to power 5, column 3 raised to power 5, column 6 raised to power 3, and the square root of the absolute value of column 9 with its original sign. This made 15 features.

The **larger feature set** added column 22 and column 2 raised to power 3. This made 17 features. I scaled these feature sets before fitting ridge. The added powers let a linear model fit some curved patterns.

Training weights and prediction limits

I gave some training rows more weight based on their target values. First I measured how far each target was above or below the training mean in standard deviation units. Its weight was 1 plus the squared distance times a tail setting. I used one setting for values below the mean and another for values above it.

The smaller ridge model used 0.2 for both tails. The larger reference used 0.5 below the mean and 0.3 above it. Both used ridge alpha 60. Alpha controls how strongly ridge shrinks its coefficients. Most predictions were limited to the training 10th and 90th percentiles. I also checked other limits.

Separate feature penalties

For the larger feature set, I adjusted two groups separately. Feature positions 12 to 15 were the added powers and square root. Positions 8, 9, 11 and 16 were selected supplied features. I divided each group by the square root of its penalty before fitting ridge. A smaller penalty gave that group less shrinkage. The main setting was 0.4 for the first group and 0.8 for the second. Positions refer to the feature order above, starting at 1.

2. Models, scores, and tuning

I compared simple regression models, trees, and blends. These are actual public scores from Kaggle. Model names describe what changed instead of using experiment numbers.

Model	Public MSE
Ridge using all 64 columns	3190.60
PCA ridge and random forest	3288.04
PCA ridge and forest averaged across folds	3276.10
Ridge with separate feature group settings	3163.78
Weighted ridge with added features	3051.87
Raw stack of 16 models	3229.11
Reference blend with small prediction changes	3013.40
Blend with feature penalty changes	2965.97
Blend with an extra tail and alpha change	2966.16
Blend with residual weights	2959.83
Earlier signed model blend	2942.52
Final signed model blend	2930.80

Adding more models did not always help. The raw stack was worse than weighted ridge. One later tail and alpha change also made the score slightly worse. I kept the best confirmed result instead of assuming each new change would improve it.

Main ranges I tried

Setting	Range or choices
Ridge on 64 columns	300 evenly spaced values on a log scale from 0.00001 to 100
PCA ridge	100 values on a log scale from 0.001 to 1000; 4 PCA components
Separate feature penalties	Power group: 0.05 to 0.85. Other group: 0.1 to 1.2. The exact grid is listed below.
Additional weighted ridge settings	Alpha: 30, 40, 80, 100. Lower tail: 0.3, 0.4, 0.5, 0.6, 0.7.
Residual weights	Cutoff: 0.75, 1, 1.5, 2, 3. Minimum weight: 0.25 or 0.5.
Small correction strength	0.1, 0.25, 0.5 or 1. Final small blend steps: 0.02 to 0.20.

The 200 setting group search used power penalties 0.05, 0.1, 0.15, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.85 and other penalties 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.85, 1, 1.2 at alpha 60. A second block used the four alphas above, powers 0.15, 0.3, 0.5, 0.7, 0.85, other penalty 0.7, and the five lower tail values above.

3. Settings for the models in the final blend

The final blend kept predictions from these models. Each model is fitted from the training data. The numbers in the first column identify the rows in the final weight table. They are not experiment versions. Unless listed otherwise, I used the defaults in the software versions on the last page.

Model	Settings
1. Weighted ridge without limits	Smaller feature set, alpha 60, both tails 0.2.
2. PCA and forest fold average	Use model 14 settings. Fit 50 times using five shuffled folds for each seed from 3000 to 3009. Use each fold's training portion. Average the predictions before limiting them to the full training target range.
3. Stack of 16 models	Settings and weights on page 5. Limit to the 5th and 95th target percentiles.
4. Ridge on 64 columns	Do not scale inputs. Choose alpha from the 300 values on page 3 using 10 fold MSE. Limit to the training target range.
5. Ridge with three extra columns	Smaller set plus columns 63, 22 and 30. Scale, alpha 55, tails 0.5 and 0.25. Limit to the 10th and 92.5th percentiles.
6. Reference prediction blend	Defined on page 7.
7. Three-group ridge	Do not scale inputs. Multiply columns 11 to 19 by 0.04 and columns 20 to 64 by 0.15. Leave first 10 unchanged. Alpha 0.06. Limit to the training range.
8. Ridge with variance weights	Larger set, alpha 60. Estimate row noise as explained on page 6.
9. Conditional model blend	Combine four models using the inputs, as explained on page 6.
10. Ridge with feature multipliers	Smaller set plus column 22. Scale, then multiply the first 11 features by 1, the next four by 1.4, 1.4, 1.4, 1.6, and the last by 0.6. Alpha 60, tails 0.3 and 0.35. Limit to 10th and 90th percentiles.
11. Ridge with tighter limits	Model 13, limited to the 12th and 88th percentiles.
12. Two-group ridge	Do not scale inputs. First 10 columns unchanged; multiply the rest by 0.11. Alpha 0.06. Limit to the training range.
13. Weighted ridge	Smaller set, alpha 60, both tails 0.2. Limit to 10th and 90th percentiles.
14. PCA ridge and forest	Scale first 10 columns, use 4 PCA components and ridge with five fold alpha selection. Ridge range is on page 3. Forest: 500 trees, depth 2, at least 6 rows per leaf, all features, seed 310. Use 65.5% ridge and 34.5% forest. Limit to the training range.
15. Polynomial change / 16. Kernel change	Defined on page 7.
17. Constant	All values equal 1. Its final weight is effectively zero.

Percentiles use the usual linear interpolation method. Scaling uses the training mean and population standard deviation. Tail settings are listed as lower tail, then upper tail. All ridge models fit an intercept unless noted.

4. Settings for the stack of 16 models

A stack combines model predictions with learned weights. Here, 10 means the first 10 columns and 64 means all columns. I scaled inputs for linear, spline, PLS, SVR and kernel models. Trees used the original first 10 columns. Weights below are rounded to eight decimal places.

Model	Weight	Settings
Weighted ridge	1.37969292	Smaller set; alpha 60; both tails 0.2; limits 10th/90th
Ridge 10	-1.43285752	Alpha 20
Spline ridge 10	0.20738783	3 knots, degree 2, no spline bias; alpha 1
PLS 10	0.60961528	2 components
SVR 10	-0.66150292	C 30; gamma 0.03; epsilon 10
Boosted trees 10	-0.04331524	90 trees; depth 1; rate 0.03; Huber loss; seed 34
Random forest 10	0.37374353	110 trees; depth 3; leaf 6; feature fraction 0.8; seed 34
Extra trees 10	-0.72413721	110 trees; depth 4; leaf 5; feature fraction 0.8; seed 34
Ridge 64	1.33006882	Alpha 60
Bayesian ridge 64	-1.22436609	Default settings
ARD ridge 64	-0.53104429	300 iterations
Lasso 64	0.15658985	Alpha 1; 10000 iterations
Elastic net 64	-0.47562560	Alpha 0.1; l1 ratio 0.5; 10000 iterations
Huber regression 10	1.59031139	Epsilon 1.35; alpha 1; 1000 iterations
Linear regression 10	-0.43456181	Default settings
Polynomial kernel 10	0.88000106	Alpha 10; degree 2; gamma 0.1; coef0 1

I used four different shuffled five fold splits to make training predictions for the stack. The random seeds were 14001, 14002, 14003 and 14004. Each row was predicted by models that had not trained on it.

I chose weights that reduced MSE while adding a penalty for moving far from the weighted ridge baseline. I tried penalty values 1, 5, 10, 20, 50, 100 and 200. The selected value was 1. Weights added to 1 and each could range from negative 2 to positive 2. SLSQP used 3000 maximum iterations and a tolerance of 0.0000000001.

For test predictions, I fitted the models on all training rows and used the fixed weights above. For the local check, I fitted the stack weights again inside each training fold. This kept the validation labels out of that fitting step.

5. Two ways to change training weights

Ridge with variance weights

I wanted to give less influence to rows that were harder to predict. I first made predictions for the training rows with the larger ridge reference. I used three shuffled folds with seed 464646 so a row was not used to fit its own prediction.

I squared each error, divided it by the mean squared error, and added 0.1. I fitted a Gamma regression model to these values. Its inputs were the first 10 columns and their squares, after scaling. Its alpha was 0.1, maximum iterations were 1000, and tolerance was 0.00000001.

For each training row, I divided the mean fitted variance by that row's fitted variance. I limited this ratio to 0.25 through 4. I multiplied the larger ridge reference's training weights by this ratio. I then fitted ridge with alpha 60 and limited predictions to the 10th and 90th percentiles.

A model blend that changes with the inputs

I used four models: the larger ridge reference, ridge with powers of the first 10 columns, SVR, and shallow boosted trees. I made predictions using three shuffled folds with seed 78310. The power model used powers 1, 2 and 3, scaled inputs, and alpha about 17.78. SVR used C 300, gamma 0.01 and epsilon 10. Boosted trees used 120 trees, depth 1, learning rate 0.03, at least 15 rows per leaf, seed 78310 and squared error loss. All four used the 10th and 90th percentile limits.

I subtracted the ridge reference prediction from each of the other three predictions. I multiplied those three differences by 1 and by scaled columns 3, 4 and 9. This made 12 new features. I scaled them and fitted ridge with alpha 1000 and no intercept to the remaining error from the reference.

For new rows, I fitted the four models on all training rows and used the same correction model. I limited its change to 20 units in either direction before adding it to the ridge reference. I then applied the 10th and 90th percentile limits again. The power model's exact alpha is 17.78279410038923.

Residual weights used in the later small change

I also used a simpler way to reduce the weight of large errors. I made three fold predictions from the main grouped ridge model, with seed 1359310. I took the absolute errors. The error scale was their median divided by 0.67448975. Each row received the smaller of 1 or 1.5 times this scale divided by its error, with a minimum weight of 0.25.

I multiplied these weights by the usual tail weights and fitted the same ridge again. I kept only the part of its prediction change that was not already covered by earlier changes. I limited that new change to 2 units before the final blend weight was applied.

6. How I made the small prediction changes

These steps use model predictions and input values, not test labels. I applied them to all 200 test rows together. When I say to remove overlap, I fit the new prediction difference to the earlier difference columns using least squares and subtract that fitted part. This keeps the part of the new change that is different.

The reference prediction blend

Start with weighted ridge using the smaller set, alpha 45, both tail settings 0.3 and limits at the 10th and 90th percentiles. Call this the alternate ridge. Make an initial blend with model 13 weight 0.474368, alternate ridge weight 0.672320, model 4 weight 0.310852, model 14 weight negative 0.080600, and model 10 weight negative 0.376940.

Take two differences from the initial blend. The first is 0.3730961449 times the difference between model 3 and the initial blend. The second is the alternate ridge minus the initial blend. The reference prediction blend, model 6, is the initial blend minus 0.8531260107 times the first difference, plus 0.3464899419 times the second.

Polynomial and kernel changes

For model 15, fit ordinary linear regression using the first 10 columns and their squares. Scale each input using its training minimum and maximum. Subtract model 6 from this prediction and remove overlap with the two differences above. Scale the remaining change so its average absolute size is 0.5. Add it to model 6.

For model 16, scale the first 10 columns and use an RBF kernel with gamma 0.01 and ridge alpha 1. Fit to targets after subtracting their training mean, then add that mean back to predictions. Subtract model 6. Remove overlap with the two earlier differences and the model 15 change. Scale it so its average absolute size is at most 1 and its largest absolute size is at most 5. Add it to model 6.

Five extra ridge changes

The settings below are written as (alpha, power penalty, other penalty, lower tail). The upper tail stays at 0.3. Each grouped ridge uses the larger feature set and limits at the 10th and 90th percentiles. The plain reference has alpha 60 and tails 0.5 and 0.3, without separate group penalties.

Change	How to make it
Group change	Grouped ridge (60,0.5,0.7,0.5) minus the plain reference.
First penalty change	(60,0.45,0.7,0.5) minus (60,0.5,0.7,0.5). Remove overlap with the group change.
Second penalty change	(60,0.4,0.8,0.5) minus (60,0.45,0.7,0.5). Remove overlap with the group change.
Tail and alpha change	(65,0.4,0.8,0.65) minus (60,0.4,0.8,0.5). Remove overlap with the group change and the sum of the first two penalty changes.
Residual weight change	Residual weighted ridge from page 6 minus (60,0.4,0.8,0.5). Remove overlap with the group change, summed penalty changes, and tail and alpha change.

For the two penalty changes and the tail and alpha change, multiply the remaining difference by the smaller of 0.25 or 0.5 divided by its largest absolute value. For the residual change, use the smaller of 1 or 2 divided by its largest absolute value. The notebook includes very small number guards to avoid dividing by zero.

7. Final weights and running the model

To make the final predictions, multiply each model or change by its weight and add the results. Negative weights subtract a prediction. These are the weights for the submitted model, rounded to eight decimal places. The notebook keeps full precision. The model numbers refer to page 4.

Model or change	Weight	Model or change	Weight
1. Weighted ridge, no limits	-0.00708649	12. Two-group ridge	0.40195971
2. PCA and forest fold average	2.09329627	13. Weighted ridge	-1.28818494
3. Stack of 16 models	0.74056079	14. PCA ridge and forest	-3.05616765
4. Ridge on 64 columns	-0.08168763	15. Polynomial change	7.31245186
5. Ridge, three extra columns	0.35766960	16. Kernel change	14.50920728
6. Reference prediction blend	-18.51485987	17. Constant	-0.00000000
7. Three-group ridge	-0.53054730	Group change	-1.09184673
8. Ridge, variance weights	-0.23939019	First penalty change	6.01691356
9. Conditional model blend	0.03188613	Second penalty change	6.01691356
10. Feature multiplier ridge	0.07376478	Tail and alpha change	-5.77127725
11. Ridge, tighter limits	-0.80287236	Residual weight change	6.36251241

Running the notebook

Use Python 3.12, NumPy 2.3.5, pandas 3.0.1, SciPy 1.18.1, scikit-learn 1.9.1 and threadpoolctl. The package list is included. Keep the data folder beside the notebook and run the cells in order. The four supplied data files are the training inputs, training outputs, test inputs and sample submission.

The notebook fits the models from the training data and creates a new prediction file. It does not load old predictions to make the new ones. It keeps the sample IDs and order, and produces 200 rows with columns Sl. No. and Output. The full test batch must be predicted together because the small changes use averages and overlap across that batch.

I checked the complete run against the file scored on Kaggle. All IDs matched. The largest prediction difference was less than 0.000001. The original scored file stays unchanged. The optional final validation step is off for the normal run because it takes longer.

The weights give a compact form of the actual submitted blend. They were calculated from existing prediction vectors to reproduce that file. Their exact values and all model settings are kept in the notebook. Rounding in this report can cause small differences, so use the notebook for the exact submitted predictions.

8. Evaluation and what I learned

Checking performance

The final public MSE was **2930.80**. The earlier signed blend scored **2942.52**, so the improvement was **11.72**. A local five fold check gave **3483.96**. The local and Kaggle checks used different rows. I did not assume that a local gain would give the same gain on Kaggle.

For the local check, I used five shuffled folds with seed 1399310. I fitted scaling, residual weights and stack weights using only the training part of each fold. The final feature choices and blend weights were still choices made during the whole project. That means this was not a completely fresh test of the search.

How I chose the last blend

The later blend search used predictions from 41 scored submissions and their reported MSE values. There is a useful relationship between the MSE of a blend and the MSE of its parts. It lets the unknown target terms cancel when the weights add to 1. I used this relationship and checks across the prediction rows to put an upper limit on the blend's public MSE, under the same scoring assumptions. This used aggregate scores, not individual hidden labels.

The calculated upper limit was about **2936.82** before submission. The actual score was **2930.80**. The last batch checked 100 small blend changes. These were changes to existing predictions, not 100 newly trained models. They used five model difference directions and positive or negative steps of 0.02, 0.04, up to 0.20 units. The earlier best blend stayed best under that check.

This check assumed the saved scores used the same public rows and metric. It did not predict the private score. I used the public leaderboard many times to choose settings. That can fit the public results too closely. I would use fewer public checks and keep a separate validation set if I did this again.

What helped and what I learned

The most useful idea was to keep a decent ridge model and change it a little. Separate penalties helped control the added features. Residual weights reduced the influence of rows with large errors. Removing overlap kept a new change from repeating an old one.

I learned that a bigger model is not always better. The raw stack had a worse public score than weighted ridge. I also learned to keep the best confirmed model, since a promising local change can still do worse on Kaggle. Saving the settings and predictions made it easier to check which model was actually submitted.

Sources and final files

I used the DS310 regression lectures and lab, the sample code guidance, and the decision tree material as starting points. The report uses the assignment handout, supplied competition data, saved model settings, reported Kaggle scores and the leaderboard screenshot. The final Canvas files are the notebook and this report. There is no notebook in the appendix.